From the Confusion Matrix to the ROC Curve

Binary classification: a practical, reproducible construction

Morten Hjorth-Jensen

October 20, 2026

The central idea

One confusion matrix gives one ROC point

For a fixed classification threshold t, a confusion matrix determines one pair $(\mathrm{FPR}(t), \mathrm{TPR}(t))$.

- A *full* ROC curve requires varying t across the classifier's scores.
- Keep the true test labels y_i and the continuous positive-class scores s_i.
- Retraining is unnecessary: only the decision rule changes. Use your final model and call it with different threesholds t for the say the sigmoid function which outputs the final value.

Binary confusion matrix

	Predicted positive	Predicted negative
Actual positive	TP	FN
Actual negative	FP	TN

$$\mathrm{TPR} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}}, \qquad \mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{FP}+\mathrm{TN}},$$

$$\mathrm{TNR} = \frac{\mathrm{TN}}{\mathrm{TN}+\mathrm{FP}}, \qquad \mathrm{FNR} = \frac{\mathrm{FN}}{\mathrm{FN}+\mathrm{TP}}.$$

$$\mathrm{TPR} = 1 - \mathrm{FNR} \text{ and } \mathrm{FPR} = 1 - \mathrm{TNR}.$$

What must be saved from the final classifier?

For each held-out test observation i, retain

$$y_i \in \{0, 1\}, \qquad s_i \in \mathbb{R},$$

where s_i is a score increasing with confidence in class 1 (often $s_i = P(y_i = 1 \mid \mathrm{x}_i)$).

Threshold rule

$$\widehat{y}_i(t) = \begin{cases} 1, & s_i \geq t, \\ 0, & s_i < t. \end{cases}$$

A sigmoid probability, neural-network logit, SVM decision score, or tree-ensemble probability can be used. Calibrated probabilities are *not* required.

Eight test observations: a worked example

Observation i	1	2	3	4	5	6	7	8
True y_i	1	0	1	1	0	0	1	0
Score s_i	.95	.85	.75	.65	.55	.45	.35	.15

At $t = 0.50$, the predicted classes are

$$\widehat{\mathrm{y}} = (1, 1, 1, 1, 1, 0, 0, 0).$$

Thus $\mathrm{TP} = 3$, $\mathrm{FP} = 2$, $\mathrm{FN} = 1$, $\mathrm{TN} = 2$, giving

$$\boxed{(\mathrm{FPR}, \mathrm{TPR}) = (2/4, 3/4) = (0.50, 0.75).}$$

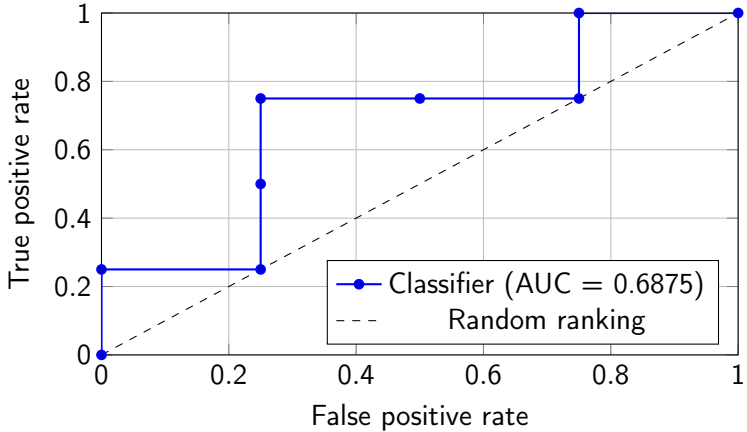
Sweep the threshold: construct a family of matrices

Threshold t	TP	FP	FN	TN	TPR	FPR
$t > .95$	0	0	4	4	0	0
.95	1	0	3	4	.25	0
.85	1	1	3	3	.25	.25
.75	2	1	2	3	.50	.25
.65	3	1	1	3	.75	.25
.55	3	2	1	2	.75	.50
.45	3	3	1	1	.75	.75
.35	4	3	0	1	1.00	.75
.15	4	4	0	0	1.00	1.00

Every row supplies one ROC coordinate

$$(\mathrm{FPR}(t), \mathrm{TPR}(t)).$$

The ROC curve for the eight observations



Why the curve is step-shaped

- Lowering t past a *positive* example increases TP and TPR, giving a vertical step.
- Lowering t past a *negative* example increases FP and FPR, giving a horizontal step.
- With tied scores, several examples can enter at once; both rates may change together.
- Thresholds above the maximum score and at or below the minimum score give the endpoints $(0, 0)$ and $(1, 1)$.

ROC axes

$$x = \mathrm{FPR}(t), \qquad y = \mathrm{TPR}(t).$$

Only a final confusion matrix?

Suppose the only stored result is

$$C = \begin{pmatrix} \mathrm{TN} & \mathrm{FP} \\ \mathrm{FN} & \mathrm{TP} \end{pmatrix} = \begin{pmatrix} 80 & 20 \\ 10 & 90 \end{pmatrix}.$$

Then

$$\mathrm{FPR} = \frac{20}{80+20} = 0.20, \qquad \mathrm{TPR} = \frac{90}{90+10} = 0.90.$$

Important limitation

The matrix supplies the point $(0.20, 0.90)$, but *not* the entire ROC curve. Recover the individual test scores from the trained model.

AUC: interpretation

Ranking probability

For an independently drawn positive score S_+ and negative score S_-,

$$\mathrm{AUC} = P(S_+ > S_-) + \tfrac{1}{2}P(S_+ = S_-).$$

- $\mathrm{AUC} = 1$: all positive observations rank above negatives.
- $\mathrm{AUC} = 0.5$: random ranking performance.
- $\mathrm{AUC} < 0.5$: scores tend to rank classes in reverse order.
- Example: $\mathrm{AUC} = 11/16 = 0.6875$ (11 correctly ordered positive–negative pairs out of 16).

AUC is not a measure of probability calibration and does not choose a decision threshold.

NumPy implementation: compute ROC points

```
import numpy as np

y_true = np.array([1, 0, 1, 1, 0, 0, 1, 0])
y_score = np.array([.95, .85, .75, .65,
                    .55, .45, .35, .15])

thresholds = np.r_[np.inf,
                   np.sort(np.unique(y_score))[::-1]]
fpr, tpr = [], []

for t in thresholds:
    pred = (y_score >= t)
    tp = np.sum((y_true == 1) & pred)
    fn = np.sum((y_true == 1) & ~pred)
    fp = np.sum((y_true == 0) & pred)
    tn = np.sum((y_true == 0) & ~pred)
    tpr.append(tp / (tp + fn))
    fpr.append(fp / (fp + tn))
```

NumPy and Matplotlib: plot ROC and AUC

```
import matplotlib.pyplot as plt

# fpr and tpr come from the preceding slide.
auc = np.trapezoid(tpr, fpr)  # NumPy 2.0+
# For older NumPy versions, use np.trapz(tpr, fpr).

plt.figure(figsize=(7, 5))
plt.plot(fpr, tpr, '-o', label=f'ROC (AUC={auc:.4f})')
plt.plot([0, 1], [0, 1], 'k--', label='Random ranking')
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.xlim(0, 1)
plt.ylim(0, 1)
plt.grid(True)
plt.legend()
plt.show()
```

Practical workflow and evaluation discipline

1. Train the classifier using training data.
2. Use validation data for model selection and threshold tuning. This is next week's exercise. Use here only the trained model, no cross-validation
3. Sort the distinct scores and sweep thresholds from high to low.
4. Compute TP, FP, FN, TN, then TPR and FPR at each threshold.
5. Plot the ROC points and compute AUC.

Take-home message

A ROC curve is a *family of confusion matrices* for one trained model, indexed by the decision threshold.

From binary classification to MNIST

Ten classes, ten scores per image

For MNIST, $y_i \in \{0, 1, \ldots, 9\}$. Keep the true label and the *entire* score vector for every held-out image:

$$\mathrm{s}_i = (s_{i0}, \ldots, s_{i9}), \qquad S \in \mathbb{R}^{N \times 10}, \qquad Y_{ik} = 1\{y_i = k\}.$$

- A softmax network supplies estimates $s_{ik} = \widehat{P}(y_i = k \mid \mathrm{x}_i)$, with $\sum_k s_{ik} = 1$.
- The final class prediction is $\widehat{y}_i = \arg\max_k s_{ik}$.
- ROC evaluation uses *scores*, not only these argmax predictions.
- There is no unique multiclass ROC curve: specify the binary reduction and averaging convention.

The 10×10 confusion matrix

Rows are true digits; columns are predicted digits:

$$C_{ab} = \sum_{i=1}^{N} 1\{y_i = a, \widehat{y}_i = b\}, \qquad a, b \in \{0, \ldots, 9\}.$$

True \ predicted	0	1	2	3	4	5	6	7	8	9
0	96	0	0	0	0	1	1	0	2	0
1	0	98	0	0	0	0	0	1	1	0
2	1	1	94	1	0	0	0	1	2	0
3	0	0	1	93	0	3	0	1	1	1
4	0	0	0	0	95	0	1	0	0	4
5	1	0	0	3	0	93	1	0	2	0
6	1	0	0	0	1	1	96	0	1	0
7	0	1	1	0	1	0	0	95	0	2
8	1	1	1	2	0	2	1	0	91	1
9	0	0	0	1	3	0	0	2	1	93

Illustrative only: 100 images per digit, $N = 1000$; not measured MNIST results.

Collapse the matrix: one digit versus all others

For class k, treat digit k as positive and the other nine digits as negative. The argmax confusion matrix gives

$$\mathrm{TP}_k = C_{kk}, \qquad \mathrm{FN}_k = \sum_{b \neq k} C_{kb},$$

$$\mathrm{FP}_k = \sum_{a \neq k} C_{ak}, \qquad \mathrm{TN}_k = N - \mathrm{TP}_k - \mathrm{FN}_k - \mathrm{FP}_k.$$

For digit 3 in the illustrative matrix:

$$\mathrm{TP}_3 = 93, \quad \mathrm{FN}_3 = 7, \quad \mathrm{FP}_3 = 7, \quad \mathrm{TN}_3 = 893,$$

$$(\mathrm{FPR}_3, \mathrm{TPR}_3) = \left(\frac{7}{900}, \frac{93}{100}\right) \approx (0.0078, 0.93).$$

One operating point per class

These rates describe the rule $\widehat{y}_i = k$. That rule generally differs from $s_{ik} \geq t$ for every choice of a single fixed threshold t. The matrix cannot recover the score-based ROC curves.

One-versus-rest (OvR): construct ten binary ROC curves

For each digit $k = 0, \ldots, 9$, form the binary evaluation problem

$$z_i^{(k)} = 1\{y_i = k\}, \qquad \widehat{z}_i^{(k)}(t) = 1\{s_{ik} \geq t\}.$$

Let $n_k = \sum_i z_i^{(k)}$. At each threshold,

$$\mathrm{TPR}_k(t) = \frac{\sum_i z_i^{(k)} 1\{s_{ik} \geq t\}}{n_k},$$

$$\mathrm{FPR}_k(t) = \frac{\sum_i (1 - z_i^{(k)}) 1\{s_{ik} \geq t\}}{N - n_k}.$$

- Sweep distinct scores in column k, from high to low.
- Include $(0, 0)$ and $(1, 1)$; process equal scores together.
- OvR is an *evaluation* method; it does not require ten separately trained classifiers. Require $0 < n_k < N$.

Per-class AUC: what question does it answer?

Digit k versus the rest

$$\mathrm{AUC}_k = P(S_k^+ > S_k^-) + \tfrac{1}{2}P(S_k^+ = S_k^-),$$

where S_k^+ is the class-k score of an image of digit k, and S_k^- is the same score for an image of any other digit.

- Integrate the ROC coordinates using the trapezoidal rule.
- A high AUC_3 means that images of 3 usually receive higher class-3 scores than images of other digits.
- Report all ten AUCs and class supports n_k to expose weak classes.
- Each OvR AUC depends on the mixture of digits in its negative group. It is not a pairwise 3-versus-8 AUC.

Micro average: pool all image–class decisions

Flatten the one-hot targets and score matrix in the *same* order:

$$(Y_{00}, Y_{01}, \ldots, Y_{N-1,9}), \qquad (s_{00}, s_{01}, \ldots, s_{N-1,9}).$$

At a common threshold t, sum the ten binary counts:

$$\mathrm{TPR}_{\mathrm{micro}}(t) = \frac{\sum_k \mathrm{TP}_k(t)}{N}, \qquad \mathrm{FPR}_{\mathrm{micro}}(t) = \frac{\sum_k \mathrm{FP}_k(t)}{9N}.$$

- There are N positive entries and $9N$ negative entries in Y.
- Integrate this *pooled* ROC to obtain $\mathrm{AUC}_{\mathrm{micro}}$.
- This ranks scores across class columns, so their scales must be comparable; softmax probabilities provide a common scale.
- Micro AUC is not, in general, an average of the ten class AUCs.

Macro average: give each digit equal weight

The macro AUC scalar

$$\mathrm{AUC}_{\mathrm{macro}} = \frac{1}{10}\sum_{k=0}^{9} \mathrm{AUC}_k.$$

To *display* a macro ROC, align the ten curves at common FPR values u and average their interpolated TPRs:

$$\overline{\mathrm{TPR}}(u) = \frac{1}{10}\sum_{k=0}^{9} \mathrm{TPR}_k(u).$$

- A finite interpolation grid gives an approximate display curve. Report the mean of the ten computed AUCs as the macro AUC.
- Support-weighted AUC is a different summary: $\mathrm{AUC}_{\mathrm{weighted}} = \sum_k (n_k/N)\mathrm{AUC}_k$.
- Equal supports imply weighted AUC = macro AUC. Micro may differ.

Which multiclass result should be reported?

Result	Construction	Main use
Confusion matrix	Argmax predictions; true rows, predicted columns	Which digits does the final rule confuse?
Per-class OvR ROC/AUC	Digit k versus the other nine digits	How well does a score rank its digit?
Macro AUC	Arithmetic mean of ten AUCs	Summary with equal weight for every digit
Micro AUC	ROC/AUC after flattening Y and S	Summary of pooled score rankings
Weighted AUC	Mean of class AUCs weighted by n_k/N	Summary emphasizing more frequent digits

A clear report

State OvR, the averaging rule, class order and test supports. Also report accuracy, the confusion matrix and per-class results.

NumPy: prepare MNIST labels, scores and counts

Use NumPy 2.0+, held-out digit labels `y_test` and `scores` of shape $(N, 10)$. The code continues across slides.

```
import numpy as np
y = np.asarray(y_test)
S = np.asarray(scores, dtype=float)
K = 10
if y.ndim != 1 or y.size == 0:
    raise ValueError("Expected a nonempty label vector")
if not np.issubdtype(y.dtype, np.integer):
    raise ValueError("Labels must be integer digits")
if np.any((y < 0) | (y >= K)):
    raise ValueError("Labels must lie in 0,...,9")
if S.shape != (y.size, K) or not np.isfinite(S).all():
    raise ValueError("Expected finite (N, 10) scores")
Y = (y[:, None] == np.arange(K))  # one-hot Boolean array
support = Y.sum(axis=0)
C = np.zeros((K, K), dtype=int)
np.add.at(C, (y, S.argmax(axis=1)), 1)
accuracy = np.trace(C) / y.size
```

NumPy: a reusable, tie-aware binary ROC function

```
def binary_roc(z, s):
    z, s = np.asarray(z), np.asarray(s, dtype=float)
    if (z.ndim != 1 or z.size == 0 or z.shape != s.shape
            or not np.all((z == 0) | (z == 1))
            or not np.isfinite(s).all()):
        raise ValueError("Expected binary labels and finite scores")
    z = z.astype(bool)
    P, Q = int(z.sum()), int((~z).sum())
    if P == 0 or Q == 0:
        raise ValueError("ROC needs positives and negatives")
    order = np.argsort(-s, kind="stable")
    s, z = s[order], z[order]
    # End indices of tied-score blocks:
    ends = np.r_[np.flatnonzero(s[:-1] != s[1:]), s.size-1]
    tp = np.cumsum(z)[ends]
    fp = 1 + ends - tp
    tpr = np.r_[0.0, tp / P]
    fpr = np.r_[0.0, fp / Q]
    thresholds = np.r_[np.inf, s[ends]]
    return fpr, tpr, thresholds
```

NumPy: per-class, micro, macro and weighted AUC

```
if np.any((support == 0) | (support == y.size)):
    raise ValueError("Each OvR problem needs both classes")
curves = [binary_roc(Y[:, k], S[:, k]) for k in range(K)]
auc_class = np.array([
    np.trapezoid(tpr, fpr) for fpr, tpr, _ in curves
])
fpr_micro, tpr_micro, _ = binary_roc(Y.ravel(), S.ravel())
auc_micro = np.trapezoid(tpr_micro, fpr_micro)
auc_macro = auc_class.mean()
auc_weighted = np.average(auc_class, weights=support)
# A display approximation; do not redefine auc_macro from it.
grid = np.linspace(0.0, 1.0, 1001)
tpr_macro = np.mean([
    np.interp(grid, fpr, tpr) for fpr, tpr, _ in curves
], axis=0)
print("digit, support, AUC")
for k in range(K):
    print(k, support[k], f"{auc_class[k]:.6f}")
print("micro, macro, weighted:", auc_micro, auc_macro, auc_weighted)
```

Matplotlib: display the ten curves and their summaries

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 6))
for k, (fpr, tpr, _) in enumerate(curves):
    ax.plot(fpr, tpr, lw=1, alpha=0.7,
            label=f"{k} vs rest: {auc_class[k]:.3f}")
ax.plot(fpr_micro, tpr_micro, lw=2.5, color="black",
        label=f"Micro: {auc_micro:.3f}")
ax.plot(grid, tpr_macro, lw=2.5, ls="--", color="magenta",
        label=f"Macro (grid): mean AUC={auc_macro:.3f}")
ax.plot([0, 1], [0, 1], "k:", label="Random ranking")
ax.set(xlabel="False positive rate", ylabel="True positive rate",
       xlim=(0, 1), ylim=(0, 1), title="MNIST: OvR ROC evaluation")
ax.grid(True, alpha=0.3)
ax.legend(loc="lower right", fontsize=8, ncol=2)
fig.tight_layout()
plt.show()
```

Each curve uses the same test images. The macro display uses interpolation; its legend reports the exact arithmetic mean of the ten per-class AUCs.

Evaluation cautions: data and score semantics

- **Keep the test set held out.** Fit preprocessing on training data; choose models, calibration and decision thresholds on validation data (next week when we do Cross-validation). Use the final test evaluation for reporting.
- **Check row alignment and column order.** Score row i must belong to label y_i; column k must mean digit k.
- **Use the original class scores.** Hard labels or their one-hot encodings discard the rankings needed for the full ROC.
- **Fix the score definition.** OvR accepts uncalibrated scores, but a network's logits and softmax probabilities can rank images differently within a class; softmax is not a column-wise transform.
- **Interpret micro AUC carefully.** Strictly increasing transforms applied separately to columns preserve each OvR AUC but may change micro AUC by changing comparisons across columns.

Evaluation cautions: what AUC can hide

- **Undefined classes.** With no positive or no negative examples, an OvR ROC/AUC is undefined. Report the issue; do not replace it by zero. The code stops rather than silently changing the ten-class average.
- **Class imbalance.** Each digit faces nine other digits. Also inspect precision–recall curves, class supports and the confusion matrix; a small FPR can still yield many false positives.
- **Operating decisions.** High AUC does not guarantee high argmax accuracy or calibrated probabilities. Independent OvR thresholds may select several digits or none; define a single-label or rejection policy.
- **Variation and uncertainty.** Compare models on the same test set. If bootstrapping uncertainty, resample whole images with all ten scores together, not individual flattened entries; retain both labels for every reported OvR problem.

Multiclass take-home workflow

1. Save $(y_i, s_{i0}, \ldots, s_{i9})$ for every held-out image.
2. Use $\arg\max_k s_{ik}$ for the 10×10 confusion matrix and accuracy.
3. For each digit, threshold its score column and compute one OvR ROC/AUC.
4. Flatten one-hot labels and scores for micro ROC/AUC.
5. Average the ten per-class AUCs for macro AUC; use an interpolated mean curve only for display.
6. Report supports, per-class performance, summary conventions and relevant decision thresholds.

The same principle extends from the binary case

A ROC curve requires a family of score-based decisions. In MNIST, first specify *which digit is positive* and then *how the ten evaluation problems are summarized*.

References for the multiclass extension

- scikit-learn: **Multiclass Receiver Operating Characteristic (ROC)**. OvR evaluation and micro/macro curve construction.
- scikit-learn: **roc_auc_score**. Macro and support-weighted AUC definitions; multiclass conventions.
- scikit-learn: **roc_curve**. Threshold rule, binary ROC coordinates and endpoints.
- TensorFlow/Keras: **MNIST dataset loader**. Digit labels and dataset description.

The NumPy implementation is self-contained and does not depend on scikit-learn. The numerical confusion matrix in this section is an illustration, not a benchmark measurement.